

A Corpus-Scale Static Analysis of Windows 10/11 System Binaries: Characterization and Cross-Build Differences

Abdelrahman Khalid 
abdnul108@gmail.com

September 2026

Abstract

Windows system binaries form a large and evolving population whose structural, security, and code-signing characteristics are difficult to observe at corpus scale. This paper presents a corpus-scale statistical characterization of 15,695 Windows system binaries spanning Windows 10 and Windows 11. The analysis examines structural composition, import and export characteristics, security-relevant configuration, and code-signing properties, enabling these characteristics to be compared across releases. The resulting measurements establish an empirical baseline for the observable structural and security properties of the Windows system-binary ecosystem and reveal how these properties vary between operating-system versions.

1 Introduction

Windows ships as a closed distribution comprising tens of thousands of executables and libraries. Despite the central role of Windows system binaries in operating-system functionality and security research, their properties are rarely studied as a population. Existing analyses commonly operate at the level of individual binaries, specific interfaces, or narrowly defined classes of artifacts. This leaves a less explored question: *what statistical and structural characteristics emerge when the Windows system-binary ecosystem is examined at corpus scale?*

Answering this question requires more than measuring a single property of individual binaries. A corpus-scale characterization must capture multiple observable aspects of the binary ecosystem, from the internal organization of individual executables to the interfaces through which they interact and the mechanisms establishing their provenance and trust. The analysis therefore considers three complementary perspectives, each addressing a distinct layer of the ecosystem: *intrinsic binary structure, inter-binary relationships, and security properties*.

Intrinsic binary structure captures the structural composition and configuration of individual binaries, including their section organization, layout, permissions, and security-relevant features. Inter-binary relationships capture how binaries consume and expose functionality through imports and exports, allowing dependencies and subsystem-level patterns to be examined across the corpus. Security properties consider binary provenance and trust through code-signing characteristics and the identities of observed signing authorities.

Viewed in this manner, corpus-scale analysis enables questions that are difficult to answer from individual-binary examination alone. At the level of structure, it becomes possible to determine which binary configurations are typical, how structural conventions are distributed, and which characteristics deviate from those conventions. At the interface level, the corpus can reveal which functions and subsystems dominate binary dependencies, how functionality is exposed through exports, and whether these patterns vary across operating-system versions. At the provenance level, the corpus provides a means of measuring the prevalence and distribution of signed components and identifying the authorities responsible for them.

The remainder of this paper details the corpus and analytical process underlying this characterization. § 2 describes the corpus scope, data collection procedure, and analysis pipeline; subsequent sections present the resulting statistics across structural, interface, and provenance dimensions and compare them across the two operating-system versions.

2 Methodology

2.1 Corpus Scope and Composition

The corpus comprises 15,695 binaries collected from Windows 10 (build 10.0.19045) and Windows 11 (build 10.0.26200) system installations, of which 63.16% are x64 and 36.83% are x86.

Binaries were collected from live, in-use Windows installations rather than a freshly provisioned VM image, to better reflect the binary composition of a real-world deployed system, including updates, driver installs, and optional components a stock VM would not accumulate.

2.2 Data Collection

System folders critical to running the Windows OS were fully included in the corpus. Folders containing per-installation or per-hardware content, such as WinSxS and DriverStore, were excluded, as they do not reflect binaries universally present across a standard Windows installation and are therefore considered out of scope for this paper. No unresolved import dependencies were found pointing exclusively to WinSxS or DriverStore-resident modules, confirming this exclusion did not compromise corpus integrity.

Since the corpus was collected from a live, in-use Windows installation rather than a clean reference image, binary state reflects the system’s operational history rather than a pristine out-of-box configuration. Some binaries – particularly signing status and presence/absence of specific drivers – may therefore reflect third-party software installation or modification after initial deployment, rather than characteristics of the base OS build as shipped. This is a property of the collection method rather than a corpus-scope decision, and is noted here as a limitation on how findings drawn from this corpus should be interpreted.

2.3 Analysis Pipeline

The analysis pipeline consists of six stages, illustrated in [Figure 1](#):

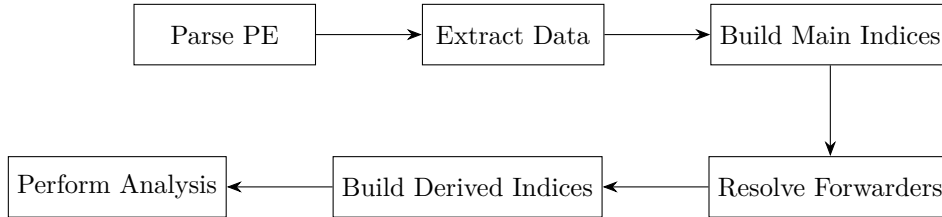


Figure 1: The six-stage analysis pipeline.

Parse PE Parses the raw PE structure of each binary, extracting headers, sections, and directory tables.

Extract Data Extracts import/export/exception tables, section/certificate/signing metadata, and load config directory information from the parsed structure.

Build Main Indices Constructs a per-binary metadata file covering data sections and load config, including imports, exports, exceptions, and relocations.

Resolve Forwarders Follows forwarder chains, including API Set contracts, to their terminal implementation.

Build Derived Indices Aggregates per-binary data into corpus-scale structures, including DLL dependencies, the hash index, and the export index, used for cross-binary queries.

Perform Analysis Computes aggregate statistics, structural anomaly detection, and cross-corpus comparisons over the derived indices, with results exposed to both text-based and graphical (e.g., Dash/Plotly) reporting tools.

The underlying parser extracts data exhaustively from each binary; no field available in the PE structure is discarded at parse time. The output schema, however, is deliberately structured around the semantics of the binary format being analyzed rather than around a generic key-value dump of parser output. This reflects a research design decision rather than a parsing limitation: by keeping the schema format-semantic rather than parser-shaped, the underlying binary format could in principle be swapped

or extended (e.g. to formats beyond PE) without requiring the schema itself to be redesigned around the new parser’s internal representation.

The six stages span three distinct implementation tiers. *Parse PE* is performed by a parsing submodule within a binary analysis SDK developed for this work, whose parsing components construct a semantic, format-normalized binary model rather than a raw structural dump. While PE is the format evaluated in this paper, this model is not PE-specific, and the SDK’s broader scope extends beyond structural parsing. *Extract Data* is performed by a subtool built on this model, which serializes the parsed corpus to disk via json-c. The remaining stages — *Build Main Indices*, *Resolve Forwarders*, *Build Derived Indices*, and *Perform Analysis* — are implemented as standalone Python scripts operating on this JSON corpus. The SDK and its associated subtool are closed-source and under active development, targeting constrained and dependency-free execution environments; this paper does not release either. The Python scripts implementing the remaining four stages, together with the underlying JSON corpus they operate on, are released alongside this paper [1]. to permit independent verification and extension of all corpus-scale statistics reported in § 3.

3 Statistics

With the corpus and extraction pipeline established in § 2, this section presents the corpus’s structural and statistical characterization. The section proceeds as follows:

- § 3.1 characterizes the structural composition of the corpus, examining section counts, naming conventions, and layout patterns;
- § 3.2 presents import statistics, examining function-level and subsystem-level trends across the corpus;
- § 3.3 presents export statistics, following an analogous structure applied to exported symbols;
- § 3.4 examines security-relevant structural and configuration features, including section permissions and load-config-derived mitigations, and discusses observed false positives;
- § 3.5 reports signature statistics, covering the prevalence of code signing and the identities of observed signing authorities;

3.1 Structural Characteristics

The Windows 10 corpus comprises 7,707 binaries (6,342 DLLs, 926 drivers, and 439 executables); the Windows 11 corpus comprises 7,988 binaries (6,548 DLLs, 961 drivers, and 479 executables), an overall increase of 3.6%. DLLs grew from 6,342 to 6,548 (+3.2%), tracking close to the aggregate rate, while drivers grew from 926 to 961 (+3.8%), modestly outpacing it. The executable population grew from 439 to 479 (+9.1%), disproportionate relative to the corpus as a whole, though its smaller base makes this percentage more sensitive to individual binary presence or absence than the driver or DLL deltas.

Sections within a PE binary partition code, data, and metadata into discrete regions, each carrying its own permissions and content semantics. A binary’s section layout is therefore a direct artifact of its toolchain and build configuration, making section-level statistics a useful baseline for characterizing the corpus prior to any behavioral or security analysis. The Windows 10 corpus comprises 46,002 sections across all binaries, with a mean section size of 78,497 bytes; the Windows 11 corpus comprises 52,324 sections, with a comparable mean of 78,036 bytes. Section naming in both builds is dominated by a small number of canonical MSVC/linker-emitted names — `.rsrc`, `.data`, `.reloc`, `.text`, and `.rdata` together account for the majority of observed sections — alongside a long tail of vendor and compiler-specific names (e.g., `fothk`, `.didat`, `GFIDS`, `.00cfg`, `_wpp.sf`) reflecting driver frameworks, control-flow-guard metadata, and toolchain-specific artifacts. Section alignment is uniform across the entire corpus, with all sections in both builds reporting an alignment of 16 bytes; this uniformity indicates a consistent linker default rather than binary-specific configuration, and is noted here for completeness rather than as a distinguishing characteristic.

Table 1 reports the presence of selected optional PE data directories across the corpus. These directories are not universally present – their inclusion depends on whether a binary exercises the corresponding functionality (e.g., delayed loading, TLS callbacks, CLR interop) – and their presence rates therefore serve as a coarse indicator of the diversity of build configurations represented in the corpus, rather than of any individual binary’s behavior.

Directory / Feature	Windows 10 (%)	Windows 11 (%)	Δ count	Δ pp
Debug directory present	7,620 (98.86%)	7,945 (99.46%)	+325	+0.60
Rich header present	7,661 (99.40%)	7,938 (99.37%)	+277	-0.03
Relocations present	7,153 (92.80%)	7,455 (93.33%)	+302	+0.53
Resource directory present	7,643 (99.17%)	7,920 (99.15%)	+277	-0.02
Import directory present	6,716 (87.13%)	6,989 (87.49%)	+273	+0.36
Export directory present	6,150 (79.80%)	6,380 (79.87%)	+230	+0.07
Delay-import directory present	3,055 (39.64%)	3,265 (40.87%)	+210	+1.23
TLS directory present	1,654 (21.46%)	1,933 (24.20%)	+279	+2.74
.NET / CLR header present	59 (0.77%)	67 (0.84%)	+8	+0.07

Table 1: Presence of selected PE data directories across analyzed Windows binaries, as count and percentage of corpus.

A pattern that emerges from Table 1 is a split between directories that tracked corpus growth and those that outpaced it. Debug, Rich header, Relocations, Resource, Import, and Export directory presence all moved in lockstep with the 3.6% aggregate growth, each shifting by less than one percentage point – consistent with a stable population simply growing in place rather than changing composition. Delay-import, TLS, and .NET/CLR header presence broke from this pattern, growing at +6.9%, +16.9%, and +13.6% respectively relative to their own Windows 10 base counts – two to five times the corpus growth rate, and the only directories in Table 1 to do so. This divergence anticipates the security-feature adoption trends detailed in § 3.4, where specific mitigations show similarly non-proportional shifts.

Having established the corpus’s structural composition in § 3.1, attention now turns to its behavioral surface, beginning with import symbols in § 3.2.

3.2 Import Statistics

Characterizing import symbols is critical to showing the dependencies of binaries upon one another. In Windows, binaries import functions through the import table statically, and the loader dynamically resolves the address or ordinal of each import to the correct library. This mechanism occurs in all PE binaries on the Windows system, including .exe, .dll, and .sys files. In Table 2, the top imported functions across both Windows versions represent functions that are either Windows OS bookkeeping functions or C runtime library functions injected by the compiler. A pattern that emerges is that no other function type occurs beyond these two categories; they are functions necessary for binaries to run, typically in userland or kernelland, and to communicate with the runtime library.

Rank	Function	Windows 10	Windows 11	Δ count
1	GetCurrentThreadId	6,116	6,347	+231
2	GetCurrentProcessId	6,107	6,345	+238
3	GetSystemTimeAsFileTime	6,088	6,326	+238
4	QueryPerformanceCounter	6,084	6,318	+234
5	GetCurrentProcess	6,073	6,140	+67
6	SetUnhandledExceptionFilter	6,022	6,123	+101
7	UnhandledExceptionFilter	6,017	6,096	+79
8	TerminateProcess	6,017	6,084	+67
9	_initterm	5,803	6,056	+253
10	memset	5,317	5,725	+408

Table 2: Top 10 imported functions in analyzed Windows binaries.

Windows consists of many subsystems that make up the Windows OS ecosystem, with additional subsystems introduced over time. One way Windows organizes these subsystems is through function name prefixes: Windows follows a strict naming convention in which a given prefix is typically owned by a single library, forming the core of that subsystem’s API. Table 3 catalogs these prefixes as observed across the corpus, spanning core kernel subsystems (e.g., Nt, Zw, Rtl, Ke), cryptography (BCrypt, NCrypt, Crypt), networking (Net, Dns, WinHttp), and other subsystems including COM, graphics, and tracing/diagnostics. This breadth illustrates the scale of the API surface a modern Windows binary may draw upon, extending well beyond user-facing Win32 functionality into direct kernel-level interaction. As functions in several of

these categories – particularly the core kernel prefixes – interact directly with kernel-mode structures, bypassing higher-level Win32 abstractions, their presence and growth represent an expanding attack surface for binaries that rely on low-level system access. Not every subsystem follows this growth trend, however: **Ob** (Object Manager) imports fall from 8,664 to 2,484 between Windows 10 and Windows 11, a decrease of 6,180 – the largest magnitude change of any prefix in the corpus. The cause of this decline is not established here and may reflect API consolidation, a shift in how these calls are counted, or another factor outside this paper’s scope.

Prefix	Subsystem / Category	Windows 10	Windows 11	Δ count
Nt	Native API (core kernel)	10,661	11,768	+1,107
Zw	Native API (core kernel)	3,368	3,871	+503
Rtl	Runtime Library (core kernel)	41,282	46,058	+4,776
Ke	Kernel (core kernel)	5,570	6,526	+956
Ex	Executive support (core kernel)	4,959	6,111	+1,152
Ob	Object Manager (core kernel)	8,664	2,484	−6,180
Mm	Memory Manager (core kernel)	1,651	1,772	+121
Cm	Config Manager (core kernel)	252	295	+43
Se	Security Ref. Monitor (core kernel)	3,868	3,935	+67
Io	I/O Manager (core kernel)	5,986	6,349	+363
Ps	Process/Thread Mgr (core kernel)	1,483	1,746	+263
Hal	Hardware Abstraction Layer	86	99	+13
Ldr	Loader (core kernel)	513	625	+112
FsRtl	Filesystem Runtime	757	845	+88
Alpc	IPC	83	86	+3
Etw	Tracing/diagnostics	5,649	5,382	−267
Perf	Tracing/diagnostics	226	339	+113
Wmi	Tracing/diagnostics	323	307	−16
BCrypt	Crypto (CNG)	5,209	5,673	+464
NCrypt	Crypto (CNG)	1,435	1,536	+101
Crypt	Crypto (legacy)	7,048	7,037	−11
Co	COM runtime	30,351	31,098	+747
Ole	OLE	1,724	1,469	−255
Rpc	RPC runtime	7,533	7,931	+398
Net	Networking	2,351	2,580	+229
Dns	Networking	491	639	+148
WinHttp	Networking	2,901	3,180	+279
NtGdi	Graphics/UI	1,080	1,067	−13
NtUser	Graphics/UI	1,125	1,360	+235
Ver	Misc	1,167	1,243	+76

Table 3: Known Windows subsystem and API-family import prefixes across analyzed Windows binaries.

Beyond categorizing imports by subsystem prefix, it is also useful to examine which specific libraries these functions are drawn from. Table 4 shows a clear pattern: the most frequently imported libraries across the corpus are overwhelmingly **api-ms-win-*** virtual DLLs, accounting for 22 of the 25 entries. These **api-ms-win-*** libraries are virtual DLLs belonging to the Windows API Set mechanism, which plays a critical role in the Windows loader. API Sets allow Microsoft to change backing DLL implementations, split monolithic DLLs, or ship edition-specific implementations (e.g., Nano Server, IoT, and historically Windows Core OS variants) without requiring applications to be recompiled against the virtual name. The mapping between virtual API Set names and their host DLLs is maintained in **apisetschema.dll**, loaded by **winload.exe** at boot time and mapped into kernel memory during kernel initialization; a copy of this schema is subsequently mapped into each user process and made accessible via the **PEB.ApiSetMap** pointer [2, 3]. As API Set resolution occurs at load time, a binary that fails to resolve, bypasses, or otherwise deviates from standard API Set behavior may indicate anomalous loader activity worth further investigation.

Rank	Library	Windows 10	Windows 11	Δ count
1	api-ms-win-core-processthreads-l1-1-0.dll	4,326	4,578	+252
2	api-ms-win-core-sysinfo-l1-1-0.dll	4,302	4,553	+251
3	api-ms-win-core-errorhandling-l1-1-0.dll	4,298	4,502	+204
4	api-ms-win-core-profile-l1-1-0.dll	4,236	4,496	+260
5	api-ms-win-core-libraryloader-l1-2-0.dll	3,976	4,317	+341
6	api-ms-win-core-synch-l1-1-0.dll	3,599	3,859	+260
7	ntdll.dll	3,594	3,798	+204
8	api-ms-win-core-handle-l1-1-0.dll	3,403	3,731	+328
9	api-ms-win-core-synch-l1-2-0.dll	3,888	3,696	-192
10	api-ms-win-core-debug-l1-1-0.dll	2,975	3,640	+665
11	api-ms-win-core-heap-l1-1-0.dll	3,251	3,606	+355
12	api-ms-win-core-localization-l1-2-0.dll	2,995	3,482	+487
13	api-ms-win-core-string-l1-1-0.dll	2,432	3,306	+874
14	api-ms-win-core-registry-l1-1-0.dll	2,878	3,071	+193
15	msvcrt.dll	4,344	3,011	-1,333
16	api-ms-win-core-processthreads-l1-1-1.dll	1,983	2,989	+1,006
17	api-ms-win-crt-runtime-l1-1-0.dll	1,372	2,961	+1,589
18	api-ms-win-crt-string-l1-1-0.dll	1,354	2,937	+1,583
19	api-ms-win-crt-private-l1-1-0.dll	1,289	2,870	+1,581
20	api-ms-win-eventing-provider-l1-1-0.dll	2,435	2,681	+246
21	api-ms-win-core-interlocked-l1-1-0.dll	1,398	2,635	+1,237
22	api-ms-win-core-threadpool-l1-2-0.dll	1,854	2,486	+632
23	msvcp_win.dll	776	1,467	+691
24	api-ms-win-core-file-l1-2-3.dll	1	204	+203
25	api-ms-win-core-com-midlproxystub-l1-1-0.dll	506	96	-410

Table 4: Top 25 libraries by number of importing binaries across analyzed Windows binaries.

Across the entire corpus, a total of 1,216,410 functions were imported in Windows 10 binaries, compared to 1,234,768 in Windows 11 binaries, a Δ of +18,358, or an increase of approximately 1.5%. Among the libraries examined, **api-ms-win-core-processthreads-l1-1-0.dll** accounts for the largest share of imported functions in the Windows 11 corpus. Figure 2 illustrates the dependency scale of this library, shown both in full (Figure 2a) and in closer detail (Figure 2b). Each string radiating outward represents a binary that depends on the library, with a green dot indicating that the binary imports it. The analysis reflects the System32 (native 64-bit) version of the library rather than its WoW64 counterpart. Notably, the library itself exhibits no outgoing dependencies of its own; every connection observed is an incoming import from another binary. This is consistent with its nature as a virtual API Set DLL, which does not implement functionality directly but is instead resolved at runtime to its actual host implementation.

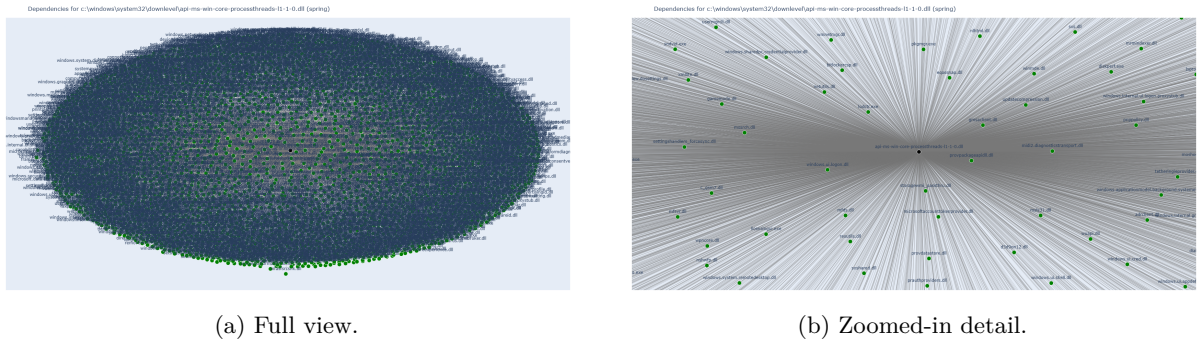


Figure 2: Dependency scale of **api-ms-win-core-processthreads-l1-1-0.dll**, the most heavily imported library in the Windows 11 corpus.

Having examined the scale and structure of import dependencies, the discussion now turns to export statistics. Each imported function must, by definition, originate as an export from some other module; comparing the volume of imports to exports across the corpus therefore highlights the asymmetry between

functions consumed and functions exposed – a distinction examined in § 3.3.

3.3 Export Statistics

While an import merely references an implementation defined elsewhere, an export is the implementation itself. Table 5 shows that the five most frequently exported functions are COM lifecycle functions, reached through COM activation rather than direct static linkage. The remaining top-ranked entries – a keyboard layout descriptor and the generic `DllMain` entry point – reflect other forms of standardized DLL interface rather than functionality-specific exports. This pattern reflects a broader convention in Windows binaries, where standardized interface functions – whether for COM activation, DLL entry, or resource description – are duplicated across many modules by necessity of the model itself, rather than the presence of distinct functionality in each.

Rank	Function	Windows 10	Windows 11	Δ count
1	<code>DllCanUnloadNow</code>	2,770	2,835	+65
2	<code>DllGetClassObject</code>	2,720	2,719	−1
3	<code>DllRegisterServer</code>	1,366	1,347	−19
4	<code>DllUnregisterServer</code>	1,352	1,326	−26
5	<code>DllGetActivationFactory</code>	759	827	+68
6	<code>KbdLayerDescriptor</code>	417	435	+18
7	<code>DllMain</code>	337	332	−5
8	<code>ServiceMain</code>	204	224	+20
9	<code>SvchostPushServiceGlobals</code>	156	166	+10
10	<code>GetProxyDllInfo</code>	129	142	+13

Table 5: Top 10 exported functions in analyzed Windows binaries.

Given that a prefix is typically owned by a single library, as established above, the export-side counts in Table 6 show where those prefix families are actually implemented, in contrast to the import-side counts in Table 3, which reflect consumption across the entire corpus. As expected, export counts for each prefix are concentrated in the small number of binaries that implement that subsystem, while the corresponding import counts are spread across every binary that depends on it.

Most prefixes track modest, proportionate growth between builds, consistent with incremental subsystem development. Two prefixes deviate sharply from this pattern and merit individual note. `Dxgk` falls from 196 to 16 exports (a 92% reduction), the largest proportional change of any prefix in the table, suggesting a substantial restructuring of the DirectX graphics kernel surface between Windows 10 and Windows 11 rather than incremental revision. `Elf`, associated with the legacy Event Log subsystem, drops from 42 exports to zero, indicating complete removal rather than gradual reduction. Unlike the aggregate export/import gap discussed below, which admits multiple non-exclusive explanations, the `Elf` case is a clean, directly attributable instance of legacy code retirement: the prefix is not merely underused but entirely absent from Windows 11.

Prefix	Subsystem / Category	Windows 10	Windows 11	Δ count
<code>Rtl</code>	Runtime Library (core kernel)	2,525	3,038	+513
<code>Nt</code>	Native API (core kernel)	1,660	1,778	+118
<code>Zw</code>	Native API (core kernel)	1,236	1,294	+58
<code>Ke</code>	Kernel (core kernel)	408	443	+35
<code>Ex</code>	Executive support (core kernel)	406	476	+70
<code>Ob</code>	Object Manager (core kernel)	214	213	−1
<code>Mm</code>	Memory Manager (core kernel)	146	147	+1
<code>Cm</code>	Config Manager (core kernel)	137	121	−16
<code>Se</code>	Security Ref. Monitor (core kernel)	1,050	1,136	+86
<code>Io</code>	I/O Manager (core kernel)	384	405	+21
<code>Ps</code>	Process/Thread Mgr (core kernel)	563	632	+69
<code>Ldr</code>	Loader (core kernel)	172	177	+5
<code>FsRtl</code>	Filesystem Runtime	225	240	+15
<code>Fs</code>	Filesystem	239	416	+177
<code>Etw</code>	Tracing/diagnostics	114	113	−1

Continued on next page

Table 6 – continued from previous page

Prefix	Subsystem / Category	Windows 10	Windows 11	Δ count
Wmi	Tracing/diagnostics	199	205	+6
BCrypt	Crypto (CNG)	304	322	+18
NCrypt	Crypto (CNG)	90	94	+4
Crypt	Crypto (legacy)	878	878	0
Co	COM runtime	1,612	1,588	-24
Ole	OLE	283	283	0
Rpc	RPC runtime	489	489	0
Net	Networking	1,262	1,380	+118
Dns	Networking	458	576	+118
WinHttp	Networking	132	174	+42
NtGdi	Graphics/UI	2,076	2,070	-6
NtUser	Graphics/UI	1,132	1,528	+396
D3DKMT	Graphics/UI	938	976	+38
Dxgk	Graphics/UI	196	16	-180
Elf	Event Log (legacy)	42	0	-42

Table 6: Known Windows subsystem and API-family export prefixes across analyzed Windows binaries.

While total import counts substantially exceed total export counts – an expected consequence of import fan-out, where a single exported function may be referenced by thousands of importing binaries – the relationship inverts at the level of unique functions, as shown in Table 7: unique exported functions outnumber unique imported functions by a factor of roughly 3:1, with approximately two-thirds of distinct exported function names across the corpus having no corresponding import anywhere else in the corpus. This gap admits several non-exclusive explanations rather than a single cause: forwarder exports that resolve to functionality implemented elsewhere; internally-consumed or ordinal-resolved APIs not visible to static import-table analysis; exports intended for consumption by user-mode or third-party software outside the scope of this corpus; and genuine legacy or compatibility-retained functions no longer referenced by any binary in the corpus. Disambiguating between these categories is left as future work; the gap is noted here as a structural characteristic of the export surface rather than attributed to any single mechanism.

Metric	Windows 10	Windows 11	Δ count
Total Imports	1,216,410	1,234,768	+18,358
Total Exports	377,959	631,673	+253,714
Unique Imported Functions	39,383 (3.24% of imports)	40,783 (3.30% of imports)	+1,400 (+0.06 pp)
Unique Exported Functions	123,429 (32.66% of exports)	123,668 (19.58% of exports)	+239 (-13.08 pp)
Unimported Exports (Gap)	84,046 (68.10% of unique exports)	82,885 (67.02% of unique exports)	-1,161 (-1.08 pp)

Table 7: Import/export totals versus unique counts across analyzed Windows binaries.

To ground this aggregate picture in a concrete case, `AcquireSRWLockExclusive` – a core Windows synchronization primitive – was traced across the full corpus to resolve its implementation, any forwarding hops, and every referencing import. Figure 3 illustrates the resulting reference graph, shown both in full (Figure 3a) and in closer detail (Figure 3b). Node color encodes the role of each entry in the trace: black denotes the queried function itself, blue denotes its implementation, orange denotes a forwarder, green denotes a calling binary, and grey denotes an unresolved or cyclic reference. The resulting graph comprises 3,623 nodes – in stark contrast to the unimported-export population discussed above, this single function’s consumption reaches a substantial portion of the corpus rather than none at all. Where Table 7 characterizes the export surface in aggregate, this trace demonstrates the same implementation-to-consumption relationship operating at the level of a single function.

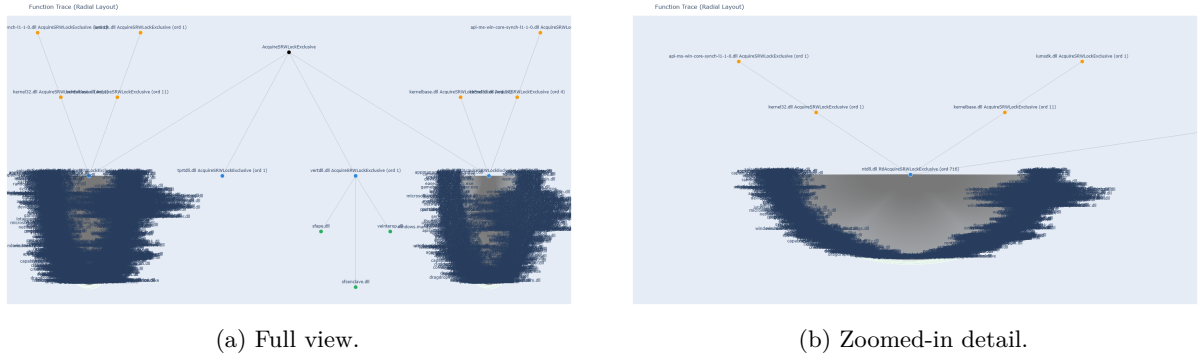


Figure 3: Reference graph of `AcquireSRWLockExclusive` across analyzed Windows 11 corpus.

Synchronization primitives such as this are also directly relevant to binary security posture, as incorrect or inconsistent locking is a well-documented source of race conditions and time-of-check-to-time-of-use vulnerabilities. Having examined the structure and scale of both the export surface in aggregate and a single function’s dependency chain in detail, the discussion now turns to the security-relevant characteristics of the corpus – a distinction examined in § 3.4.

3.4 Security-Relevant Statistics

Windows remains one of the most widely deployed operating systems, used across both enterprise and consumer contexts, which makes the security posture of its system binaries a matter of broad practical relevance rather than purely academic interest. As a closed-source platform, Windows binaries are not directly auditable by external researchers or defenders in the way open-source software is; source-level review, static analysis tooling built around source, and community code audit are unavailable by default. This is precisely what motivates corpus-scale binary analysis as a methodology: if independent verification of security posture is not possible at the source level, it must instead be reconstructed from the compiled artifact itself, at a scale large enough to be representative rather than anecdotal.

Windows is also a persistent and high-value target for attackers, and this sustained pressure has driven continuous investment in platform-level security mitigations over time. However, the existence of improved security mechanisms at the platform level does not guarantee their uniform adoption across all previously and currently shipped system binaries – mitigations are typically opt-in at compile time, and adoption depends on when and how a given binary was last built, rather than being retroactively applied. This raises a direct empirical question: across the corpus, is the presence of standard exploit mitigations – Control Flow Guard, Guard EH continuation tables, high-entropy ASLR, stack cookies, SafeSEH, and force-integrity signing – consistent across builds, or does adoption vary in ways that reveal which binaries were never rebuilt against current compiler defaults? Table 8 answers this directly, reporting adoption counts for each mitigation across the Windows 10 and Windows 11 corpora.

Mitigation	Windows 10 (%)	Windows 11 (%)	Δ count	Δ pp
Guard EH continuation table	2,979 (38.7%)	3,903 (48.9%)	+924	+10.2
Control Flow Guard (CFG)	6,725 (87.3%)	7,335 (91.8%)	+610	+4.6
CFG function table	6,236 (80.9%)	6,486 (81.2%)	+250	+0.3
High-entropy ASLR	4,648 (60.3%)	4,987 (62.4%)	+339	+2.1
Security cookie (/GS)	6,522 (84.6%)	6,859 (85.9%)	+337	+1.2
ASLR (dynamic base)	7,597 (98.6%)	7,925 (99.2%)	+328	+0.6
DEP / NX	7,528 (97.7%)	7,852 (98.3%)	+324	+0.6
SafeSEH	2,094 (27.2%)	2,069 (25.9%)	−25	−1.3
Force integrity signing	154 (2.0%)	195 (2.4%)	+41	+0.4
CFG address-taken table	4,428 (57.5%)	4,293 (53.7%)	−135	−3.7
CFG long-jump table	184 (2.4%)	37 (0.5%)	−147	−1.9

Table 8: Adoption of exploit mitigations across analyzed Windows binaries.

The data indicate that adoption is not uniform, but it is directionally consistent: eight of the eleven mitigations in Table 8 increased between Windows 10 and Windows 11. The remaining three – SafeSEH (−1.3 pp), the CFG address-taken table (−3.7 pp), and the CFG long-jump table (−1.9 pp) – all declined, consistent with their status as legacy or narrow-scope mechanisms superseded by broader CFG adoption rather than ones still under active investment. This confirms that platform-level security improvements are, in aggregate, propagating into the shipped binary corpus across builds.

Mitigation presence, however, only characterizes what protections a binary carries – it does not capture whether a binary’s own structural properties actively work against those protections. This distinction is most visible at the section-permission level. Table 9 presents the corpus-wide shift in section permission flags between Windows 10 and Windows 11.

Permission	Windows 10	Windows 11	Δ count
r-x (read/execute)	8,593 (18.7%)	12,943 (24.7%)	+4,350
r- (read-only)	27,267 (59.3%)	28,511 (54.5%)	+1,244
rw- (read/write)	10,093 (21.9%)	10,850 (20.7%)	+757
rwX (read/write/execute)	47 (0.10%)	19 (0.04%)	−28
−x (execute-only)	2 (0.00%)	1 (0.00%)	−1

Table 9: Section permission distribution shift across analyzed Windows binaries.

Read/execute sections grew both in absolute count and as a proportion of the corpus, while read/write and, notably, the RWX (read-write-execute) and execute-only categories all contracted. This is consistent with a general hardening trend: the corpus is shifting toward stricter separation between writable and executable memory. Windows enforces strict memory permission policies at the section level, including scrutiny of pages marked simultaneously writable and executable, a configuration broadly discouraged as it undermines $W \oplus X$ (write XOR execute) guarantees and expands the attack surface available to memory-corruption exploitation. Despite the broader hardening trend, RWX-flagged sections persist within the corpus: across the analyzed builds, RWX section counts decreased from 47 in Windows 10 to 19 in Windows 11, a reduction driven almost entirely by the elimination of a user-mode cluster ($13 \rightarrow 0$), while RWX sections in kernel-mode drivers persisted at a reduced but still notable rate ($34 \rightarrow 19$). Drivers retaining an RWX-flagged INIT section in Windows 11 include a mix of storage, human-interface, and hardware-access controllers, alongside several drivers – including `inpoutx64.sys` and a cluster of profiling/sampling drivers – that newly appear in this configuration in Windows 11 despite being absent from the Windows 10 RWX set.

At the opposite end of the permission spectrum, execute-only (−x) sections – the most restrictive configuration possible, granting neither read nor write access to code memory – remain vanishingly rare across both builds. Only two such sections were observed in Windows 10, belonging to `MaxxAudioEQ64.dll` and `msvcr100.dll`; by Windows 11, only the `msvcr100.dll` instance persists. Given their rarity, these do not represent a meaningful corpus-wide trend, but their near-total absence underscores that execute-only protection remains largely unadopted in practice, even as adjacent read/execute permissions expand.

A related structural signal is the presence of the Rich header, an undocumented but well-known metadata block embedded by the Microsoft Visual C++ linker that records toolchain and build environment information. Across the corpus, 46 binaries in Windows 10 and 50 in Windows 11 were missing a Rich header, with 19 binaries common to both sets, for a Δ of +4. While Rich header absence is not itself a security mitigation, it is a useful provenance signal: its absence can indicate non-standard build tooling, deliberate stripping, or a non-Microsoft linker, any of which merit closer inspection in binaries otherwise expected to follow standard Microsoft build conventions.

Finally, Extended Flow Guard (XFG) support was not observed in the load-config table for any binary across either corpus. This measurement, however, only reflects XFG-related fields historically exposed through the load config directory; Windows has since migrated some flow-guard configuration into internal CFG table structures and descriptors not captured by this pass, so the true prevalence of XFG adoption may be understated and is not conclusively characterized here.

Taken together, these findings show that while platform-level mitigation adoption and section permission hygiene are both trending toward stricter enforcement across builds, verification of a binary’s origin and integrity is a separate concern entirely. § 3.5 examines this dimension directly, reporting the prevalence and characteristics of code signing across the corpus.

3.5 Signature Statistics

Verifying a system binary is crucial to maintaining the integrity of the Windows ecosystem, preventing tampering or the loading of unauthorized code. Microsoft has evolved its verification model considerably — from embedded Authenticode signatures to catalog-based signing, allowing trust to be established without modifying the binary itself. Table 10 shows that the vast majority of system binaries carry a valid signature. Unsigned binaries decline from 14 (0.2%) in Windows 10 to 5 (0.1%) in Windows 11, a Δ of -9 , suggesting improved coverage of Microsoft’s signing pipeline across the build. A separate category of four Windows 11 binaries fail trust chain validation entirely rather than lacking a signature outright, discussed below.

Metric	Windows 10	Windows 11	Δ count
Signed (valid)	7,693 (99.8%)	7,979 (99.9%)	+286
Unsigned	14 (0.2%)	5 (0.1%)	-9
Untrusted root	0 (0.0%)	4 (0.1%)	+4
Microsoft-signed	7,520	7,895	+375
Third-party signed	173	84	-89

Table 10: Digital signature summary for analyzed Windows binaries.

Four binaries (`DMRCDecoder.dll`, `HrtfDspCpu.dll`, both architectures) fail trust chain validation with `CERT_E_UNTRUSTEDROOT`. This is not evidence of tampering; rather, these binaries are signed via Microsoft’s Artifact Signing service (formerly Azure Code Signing, then Trusted Signing), whose Public Trust root CA may not yet be present in this build’s (10.0.26200) default trust store [4].

Beyond the binary signed/unsigned split, the identity of the signer itself is informative. Table 11 enumerates every distinct signer observed across both builds. Microsoft accounts for the overwhelming majority of signatures once its several publisher identities — `Microsoft Windows`, `Microsoft Corporation`, `Microsoft Windows Publisher`, and related component-specific certificates — are considered together, consistent with the Microsoft-signed totals in Table 10. The `Microsoft Windows` identity alone grows from 7,030 to 7,403 binaries (+373), while `Microsoft Corporation` declines from 385 to 363 (-22), suggesting a shift in which internal publisher identity is used to sign core system components between builds rather than a change in overall Microsoft coverage.

The third-party signer population, by contrast, contracts sharply. Several vendors present in Windows 10 — `Dolby Laboratories`, `Skype Software Sarl`, `LunarG`, `SRS Labs`, `DTS`, and `TOSHIBA CORPORATION` among them — do not appear at all in the Windows 11 sample, reflecting the removal or replacement of the third-party audio, driver, and OEM utility binaries they signed. This mirrors the -89 count decline in third-party signed binaries noted above. It is worth noting that this dataset was captured from a live, in-use OS installation rather than a clean or minimal image; the specific third-party signers observed — audio codecs, chipset drivers, OEM utilities, and similar — are therefore a function of this particular machine’s hardware and installed software, and are not representative of a stock Windows install. A different machine, with different peripherals and drivers, would likely surface a different set of third-party signers entirely. A small number of new signers appear only in Windows 11, including `I3D Technology Inc.`, `Acer Incorporated`, `Sunplus Innovation Technology Inc.`, `Incredibuild Software Ltd.`, and `Red Fox UK Limited`, indicating newly introduced components rather than signer continuity across the build transition.

Signer	Windows 10	Windows 11	Δ count
Microsoft Windows	7,030	7,403	+373
Microsoft Corporation	385	363	-22
Microsoft Windows Software Compatibility Publisher	51	77	+26
NVIDIA Corporation	30	33	+3
Realtek Semiconductor Corp	24	–	-24
Microsoft Windows Publisher	21	31	+10
Intel Corporation	21	30	+9
Microsoft Windows Hardware Compatibility Publisher	17	1	-16
Dolby Laboratories, Inc.	15	–	–
Skype Software Sarl	10	–	–
LunarG, Inc.	8	–	–

Continued on next page

Table 11 – continued from previous page

Signer	Windows 10	Windows 11	Δ count
Future Technology Devices International Ltd	7	–	–
Microsoft Windows Hardware Abstraction Layer Publisher	6	8	+2
Microsoft Windows Kits Publisher	4	4	0
SRS Labs, Inc.	4	–	–
TOSHIBA CORPORATION	4	–	–
DTS, Inc.	3	–	–
Intel(R) Wireless Connectivity Solutions	3	–	–
Waves Inc.	3	–	–
Synopsys, Inc.	3	–	–
ASUSTeK Computer Inc.	3	–	–
Samsung Electronics CO., LTD.	3	3	0
I3D Technology Inc.	–	3	–
Microsoft 3rd Party Application Component	2	3	+1
GENESYS LOGIC, INC.	2	–	–
HP Inc.	2	–	–
Intel Corporation-Wireless Connectivity Solutions	2	–	–
Apple Inc.	2	–	–
Microsoft Windows 3rd party Component	2	2	0
Synaptics Incorporated	2	–	–
Microsoft Windows Third Party Application Component	–	2	–
Andrea Electronics	1	–	–
Real Sound Lab SIA	1	–	–
Fortemedia Inc.	1	–	–
KNOWLES ELECTRONICS LLC	1	–	–
US Waves Inc.	1	–	–
Razer USA Ltd.	1	1	0
Sony Corporation	1	–	–
Microsoft Windows Component Publisher	1	–	–
Oracle America, Inc.	1	5	+4
Sonic Focus, Inc.	1	–	–
ASUSTek Computer Inc.	1	–	–
ELAN Microelectronics Corporation	1	–	–
Intel Corporation - Client Components Group	1	1	0
Intel(R) Rapid Storage Technology	1	–	–
Intel(R) Software	1	–	–
Krisp Technologies, Inc.	1	–	–
Nmap Software LLC	1	1	0
Samsung Electronics Co., Ltd.	1	–	–
Intel(R) Embedded Subsystems and IP Blocks Group	1	–	–
Microsoft Windows Early Launch Anti-malware Publisher	1	1	0
Acer Incorporated	–	1	–
Sunplus Innovation Technology Inc.	–	1	–
Incredibuild Software Ltd.	–	1	–
Red Fox UK Limited	–	1	–

Table 11: Digital signature publishers observed on representative live Windows installations.

Together, these results close out the structural, import, export, and security-relevant statistics outlined at the start of this section, completing the characterization of the corpus described in § 2. Where this section has focused on aggregate trends and dataset-wide comparisons, § 3 turns to the anomalies these statistics help surface — individual binaries whose structure, imports, exports, or signatures depart from the patterns established here, examined in closer detail.

4 Limitation

This study is scoped to static analysis; no dynamic or runtime validation was performed, as the paper’s methodology was designed around static corpus-scale analysis rather than behavioral observation.

Corpus coverage was constrained by resource and environment availability rather than methodology: only one build each of Windows 10 and Windows 11 was used as samples, reflecting the systems accessible during this study. Testing across multiple builds per version—to account for binary changes introduced by cumulative updates and patches—was not feasible within this study’s resource constraints. Kernel-level debugging and dynamic analysis were not performed, primarily due to hardware constraints limiting the ability to run the virtual machine environments such analysis would require.

The corpus’s JSON representation is semantic rather than a literal structural mirror of the PE format—it reflects the underlying SDK’s internal binary model rather than raw PE structure field-for-field. This representation was a deliberate design choice to support the platform’s broader goals, but readers directly comparing corpus entries against raw PE structures should be aware of this distinction.

Finally, while this study and the tooling underlying it are Windows-focused, the platform is designed with future extensibility toward other operating systems in mind, should analogous analysis prove useful there.

5 Related Work

Corpus-scale Windows binary and driver analysis. Recent work has begun treating the Windows driver surface as an object of study in its own right rather than a target for one-off vulnerability discovery. Bommarito’s Windows IOCTL Census [5] recovers the dispatch and control-code surface of 27,087 signed drivers through a single architecture-neutral static pass, persisting the recovered structure – dispatch routines, decoded control codes, call graphs, and taint sinks – as a queryable database rather than a one-shot finding list. A companion effort, Needles at Scale [6], addresses target selection across a whole-OS Windows binary corpus, framing analyst attention itself as the binding constraint at scale. Both share this study’s corpus-scale orientation and its emphasis on persisting structural facts rather than emitting isolated findings, though they operate over the IOCTL dispatch and target-selection surfaces specifically, whereas this study characterizes structural and security-relevant properties across the broader system binary corpus.

Cross-platform methodological precedent. The corpus-scale, file-level approach to binary analysis applied in this study parallels prior work on Linux systems. Prokhorenko et al. [7] propose DepEx, a framework for extracting inter-package dependency relations at the level of individual executable files and dynamically loadable libraries, and apply it across 84 consecutive Ubuntu releases spanning 2005–2023 to characterize the evolution of package-level dependency structure. Both studies share a methodological commitment to systematic, file-level analysis across a large corpus of OS-distributed binaries rather than manual inspection of individual samples. The two diverge sharply in target: DepEx characterizes inter-package dependency evolution across an entire Linux distribution’s release history, whereas this study characterizes structural and security-relevant properties of individual Windows PE binaries across two builds. The disjoint operating systems, binary formats (ELF/package metadata versus PE), and object of analysis (dependency graphs versus binary structural characteristics) mean the two studies surface largely non-overlapping findings, but the shared file-level, corpus-scale methodology suggests such approaches generalize usefully across operating systems and binary ecosystems – a direction this study’s own tooling is designed to support (see Limitations, § 4).

Driver trust and signing infrastructure. This study’s discussion of code-signing prevalence and validity builds on documented Windows trust infrastructure, including Authenticode signing [8].

6 Conclusion and Future Work

None of the trends reported in § 3 are visible from a single binary. The growing structural role of API Set forwarding, the divergence between individual mitigation trends, and the resolution of clustered CERT_E_UNTRUSTEDROOT failures into a PKI migration gap rather than tampering – each of these is unremarkable, or even illegible, at the level of one binary examined in isolation. Each becomes a coherent, measurable pattern only once the corpus is treated as the unit of analysis.

This is not simply a matter of scale making individually-visible trends easier to count. A synthetic or freshly provisioned reference image would not have surfaced these patterns with the same fidelity, since several of them are plausibly artifacts of real-world deployment history – accumulated updates, driver installs, optional components – rather than properties of a stock build. Corpus-scale analysis of a live, in-use system is therefore not a scale-up of single-binary analysis; it is the vantage point from which these particular findings become visible at all.

This corpus-wide visibility is also what makes the next stage of this work possible. The same deviation-from-baseline logic that surfaces aggregate trends can be turned toward detecting individual outliers: structurally atypical binaries, clusters of unexpectedly unsigned components, and specific binaries warranting closer manual analysis, using the corpus-wide baseline established here as the detection signal. This anomaly-detection work is already underway as a direct continuation of the present study, rather than a separate undertaking. Two further extensions follow from the scope of this paper: tracking multiple builds within a single Windows version, across cumulative updates rather than only major versions, to test whether the trends identified here hold continuously or are artifacts of comparing two specific builds; and applying the same pipeline to future Windows releases, or to other closed-source operating system distributions, to test how far these findings generalize beyond this particular pair of builds.

References

- [1] Abdelrahman Khalid. *win10-11-binary-corpus-research*. Research repository for this study (analysis scripts and supporting tooling). 2026. URL: <https://github.com/OxNull111/win10-11-binary-corpus-research> (visited on 08/05/2026).
- [2] Sébastien Renaud. *Runtime DLL Name Resolution: ApiSetSchema – Part I*. Quarkslab. Apr. 2012. URL: <https://blog.quarkslab.com/runtime-dll-name-resolution-apisetschema-part-i.html> (visited on 07/19/2026).
- [3] Lucas Georges. “DLL Shell Game and Other Misdirections: The Windows Native Loader is a Magician”. In: *Symposium sur la sécurité des technologies de l’information et des communications (SSTIC)*. Synacktiv, 2019. URL: https://www.sstic.org/media/SSTIC2019/SSTIC-actes/dll_shell_game_and_other_misdirections/SSTIC2019-Article-dll_shell_game_and_other_misdirections-georges.pdf (visited on 07/19/2026).
- [4] Microsoft. *Artifact Signing Trust Models*. Service formerly known as Azure Code Signing, then Trusted Signing. Microsoft Learn. 2026. URL: <https://learn.microsoft.com/en-us/azure/artifact-signing/concept-trust-models> (visited on 08/08/2026).
- [5] Michael J. Bommarito II. *The Windows IOCTL Census: A Corpus-Scale, Multi-Architecture Database of the Driver Control-Code Surface*. June 2026. arXiv: [2606.07732](https://arxiv.org/abs/2606.07732) [cs.SE].
- [6] Michael J. Bommarito II. *Needles at Scale: LLM-Assisted Target Selection for Windows Vulnerability Research*. May 2026. arXiv: [2606.01364](https://arxiv.org/abs/2606.01364) [cs.SE].
- [7] Victor Prokhorenko, Chadni Islam, and Muhammad Ali Babar. “Analyzing the Evolution of Inter-package Dependencies in Operating Systems: A Case Study of Ubuntu”. In: *Software Architecture: 17th European Conference (ECSA 2023)*. Springer, Sept. 2023. DOI: [10.1007/978-3-031-42592-9_16](https://doi.org/10.1007/978-3-031-42592-9_16).
- [8] Microsoft. *Authenticode Digital Signatures – Windows Drivers*. Microsoft Learn. 2025. URL: <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/authenticode> (visited on 07/28/2026).